🚀 Подробный разбор всех этапов DevOps проекта

Автор: Vitalii Zaburdaiev
Проект: Health Dashboard
Курс: DevOpsUA6
Актуальный IP: `18.156.160.162`
EC2 Instance: `i-059c8320d831be2bf`

📚 Содержание

1. Приложение (Flask Application)
2. Docker Compose
3. CI/CD (GitHub Actions + Ansible)
4. Terraform (Infrastructure as Code)
5. Ansible (Configuration Management)
6. Kubernetes (Container Orchestration)
7. Monitoring (Prometheus + Grafana)
8. Testing (Unit Tests)
9. Nginx (Reverse Proxy)
10. Disaster Recovery (Infrastructure Recovery)

---

1. Приложение (Flask Application)

📁 Файл: `app/app.py`

🎯 Что это?

Flask приложение — веб-сервис для мониторинга здоровья системы. Собирает метрики (CPU, память, диск) и предоставляет их через REST API и Prometheus-совместимый `/metrics` endpoint.

🏗️ Из чего состоит приложение

```
app/
├── app.py          # Основное Flask приложение (~360 строк)
├── __init__.py     # Python package init
└── wsgi.py         # WSGI entry point (gunicorn)
```

🔍 Ключевые компоненты `app.py`

```
╔═══════════════════════════════════════════════════════════════════╗
║                    FLASK APPLICATION STRUCTURE                    ╒
╟═══════════════════════════════════════════════════════════════════╢
│                                                                   │
│  1. IMPORTS & DEPENDENCIES                                        │
│     ├── Flask (web framework)                                     │
│     ├── psutil (system metrics: CPU, RAM, disk)                   │
│     ├── psycopg2 (PostgreSQL driver)                              │
│     ├── redis (Redis client)                                      │
│     └── prometheus_client (metrics export)                        │
│                                                                   │
│  2. LOGGING (JSON format)                                         │
│     └── Structured logs: timestamp, level, message                │
│                                                                   │
│  3. PROMETHEUS METRICS                                            │
│     ├── REQUEST_COUNT (счётчик запросов по endpoint)              │
│     ├── REQUEST_LATENCY (гистограмма времени ответа)              │
│     └── Системные метрики (CPU, RAM, disk)                        │
│                                                                   │
│  4. DATABASE LAYER                                                │
│     ├── get_db_connection() → PostgreSQL                          │
│     ├── init_db() → создание таблиц                               │
│     └── save_metrics() → запись метрик в БД                       │
│                                                                   │
│  5. CACHE LAYER                                                   │
│     ├── get_cached_metrics() → чтение из Redis                    │
│     └── set_cached_metrics() → запись в Redis (TTL=10s)           │
│                                                                   │
│  6. ENDPOINTS                                                     │
│     ├── GET /           → главная страница (index)                │
│     ├── GET /health     → health check (JSON)                     │
│     ├── GET /metrics    → Prometheus метрики                      │
│     └── GET /api/system-info → подробные системные метрики        │
│                                                                   │
│  7. MIDDLEWARE                                                    │
│     ├── before_request → start timer + increment counter          │
│     └── after_request → record latency                            │
│                                                                   │
╙═══════════════════════════════════════════════════════════════════╜
```

📋 Endpoints и что они делают

Endpoint	Метод	Описание	Пример ответа
`/`	GET	Главная страница	HTML с системной информацией
`/health`	GET	Health check	`{"status": "healthy", "cpu": 12.5, "memory": 45.2}`
`/metrics`	GET	Prometheus метрики	`request_count_total{...} 42`
`/api/system-info`	GET	Полные системные метрики	JSON с CPU, RAM, disk, uptime

🔄 Как работает приложение

```
Пользователь → http://18.156.160.162/health
                        │
                        ▼
                  ┌─────────────┐
                  │   Nginx     │   :80
                  │  (proxy)    │
                  └─────────────┘
                        │
                        ▼
                  ┌─────────────┐
                  │   Flask     │   :5000
                  │   app.py    │
                  └──┬──────┬───┘
                     │      │
              ┌──────┘      └──────┐
              ▼                     ▼
        ┌──────────┐          ┌──────────┐
        │PostgreSQL│          │  Redis   │
        │  :5432   │          │  :6379   │
        │ (данные) │          │ (кэш)    │
        └──────────┘          └──────────┘
```

🎯 Что сказать на защите

> “Приложение написано на Flask — легковесный Python-фреймворк. Оно собирает метрики системы (CPU, RAM, диск) через библиотеку psutil, хранит историю в PostgreSQL, кэширует результаты в Redis для быстрого доступа, и экспортирует метрики для Prometheus. Есть health check endpoint, который используется Docker и CI/CD для проверки работоспособности.”

---

2. Docker Compose

📁 Файл: `docker-compose.yml`

🎯 Что это?

Docker Compose — инструмент для запуска многоконтейнерных приложений. Один YAML файл описывает все 6 сервисов, их сети, тома, и зависимости.

🏗️ Архитектура стека (6 сервисов)

```
┌──────────────────── docker-compose.yml ────────────────────┐
│                                                            │
│  ┌──────────────────────────────────────────────────────┐  │
│  │                   app-network (bridge)               │  │
│  │                                                      │  │
│  │  ┌──────────┐  ┌──────────┐  ┌────────────────────┐  │  │
│  │  │ postgres │  │  redis   │  │     app (Flask)    │  │  │
│  │  │ :5432    │  │  :6379   │  │     :5000          │  │  │
│  │  │ 15-alpine│  │ 7-alpine │  │ health-dashboard-app │  │  │
│  │  └──────────┘  └──────────┘  └────────────────────┘  │  │
│  │                                        │             │  │
│  │                                        │             │  │
│  │  ┌──────────┐  ┌──────────┐  ┌────────▼───────────┐  │  │
│  │  │prometheus│  │ grafana  │  │       nginx        │  │  │
│  │  │ :9090    │  │  :3000   │  │       :80          │  │  │
│  │  │ prom/... │  │ 10.4.7   │  │       alpine       │  │  │
│  │  └──────────┘  └──────────┘  └────────────────────┘  │  │
│  │                                                      │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  Volumes: postgres_data, prometheus_data, grafana_data     │
└────────────────────────────────────────────────────────────┘
```

🔍 Разбор каждого сервиса

Сервис	Образ	Порт	Роль	Лимит памяти
postgres	`postgres:15-alpine`	5432 (внутренний)	База данных для истории метрик	—
redis	`redis:7-alpine`	6379 (внутренний)	Кэш для быстрого ответа	—
app	`build: .`	5000 (внутренний)	Flask приложение	256m
nginx	`nginx:alpine`	**80 (внешний)**	Reverse proxy	64m
prometheus	`prom/prometheus:latest`	**9090 (внешний)**	Сбор метрик	128m
grafana	`grafana/grafana:10.4.7`	**3000 (внешний)**	Визуализация метрик	128m

📋 Важные особенности

- **Порты наружу:** только nginx(:80), prometheus(:9090), grafana(:3000)
- **Внутренняя сеть:** `app-network` (bridge) — контейнеры общаются по имени сервиса
- **Тома (volumes):** данные PostgreSQL, Prometheus, Grafana переживают перезапуск
- **Health check:** app проверяется каждые 30 секунд
- **Зависимости:** app ждёт postgres и redis, nginx ждёт app, grafana ждёт prometheus
- **Переменные:** из `.env` файла (пароли, URL базы данных)
- **Grafana credentials:** задаются через `GF_SECURITY_ADMIN_USER` / `GF_SECURITY_ADMIN_PASSWORD` из `.env`

📋 Основные команды

```
docker compose up -d --build     # Запуск с пересборкой
docker compose down              # Остановка и удаление контейнеров
docker compose ps                # Статус контейнеров
docker compose logs app          # Логи Flask приложения
docker compose pull              # Скачать свежие образы
```

🎯 Что сказать на защите

> “Docker Compose управляет 6 сервисами: Flask-приложение, PostgreSQL для хранения данных, Redis для кэширования, Nginx как reverse proxy, Prometheus для сбора метрик и Grafana для визуализации. Все сервисы работают в одной bridge-сети и общаются по именам контейнеров. Данные хранятся в volumes и переживают перезапуск.”

---

3. CI/CD (GitHub Actions + Ansible)

📁 Файл: `.github/workflows/ci-cd.yml`

🎯 Что это?

CI/CD пайплайн — автоматизирует тестирование, сборку и деплой при каждом push в `main`. Для деплоя используется **Ansible** — профессиональный инструмент управления конфигурацией.

🏗️ Структура pipeline (3 Stage)

```
┌──────────────────────────────────────────────────────────┐
│                      CI/CD PIPELINE                      │
│                                                          │
│  TRIGGER: push to main / PR to main                      │
│                                                          │
│  ┌──────────────────┐                                    │
│  │ Stage 1: TEST    │  Всегда                            │
│  │ pytest + cov     │                                    │
│  └──────────────────┘                                    │
│           │ ✅ PASS                                      │
│           ▼                                              │
│  ┌──────────────────┐                                    │
│  │ Stage 2: BUILD   │  Только main                       │
│  │ Docker image     │                                    │
│  │ → Docker Hub     │                                    │
│  └──────────────────┘                                    │
│           │                                              │
│           ▼                                              │
│  ┌──────────────────┐                                    │
│  │ Stage 3: DEPLOY  │  Только main                       │
│  │ Ansible playbook│                                    │
│  │ → EC2 server     │                                    │
│  └──────────────────┘                                    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

🔍 Разбор каждого этапа

Stage 1: 🧪 Run Tests

```
1. Checkout code                    # git clone
2. Set up Python 3.11               # Установка Python
3. Install dependencies             # pip install -r requirements.txt
4. Run pytest                       # pytest tests/ -v --cov=app
```

- Запускается для **всех веток** (main и PR)
- Если тесты FAIL → пайплайн **ОСТАНАВЛИВАЕТСЯ**
- Измеряет code coverage (покрытие кода тестами)

Stage 2: 🐳 Build and Push Docker Image

```
1. Checkout code                    # git clone
2. Login to Docker Hub              # docker login
3. Build and push image             # docker build + push
   Tags:
   ├── username/health-dashboard:latest
   └── username/health-dashboard:<git-sha>
```

- Запускается **только для main** (не для PR)
- Зависит от Stage 1 (`needs: test`)
- Образ пушится в Docker Hub с двумя тегами

Stage 3: 🚀 Deploy via Ansible

```
1. Checkout code                    # Для доступа к ansible/ файлам
2. Set up Python                    # Для Ansible
3. Install Ansible                  # pip install ansible
4. Configure SSH key                # Сохраняем ключ из secrets
5. Generate Ansible inventory       # Динамический inventory с IP
6. Run Ansible Playbook             # ansible-playbook playbook.yml
7. Post-deploy verification         # curl health/grafana/prometheus
```

Что делает Ansible на сервере:

```
ansible-playbook playbook.yml
│
├── Role: docker
│   ├── Install Docker + pip
│   ├── Start Docker service
│   ├── Add ec2-user to docker group
│   └── Install Docker Compose (if needed)
│
├── Role: app
│   ├── Install git
│   ├── Create /opt/health-dashboard
│   ├── git clone/pull repository
│   ├── Copy .env from .env.example
│   ├── docker compose down
│   ├── docker compose pull --ignore-buildable
│   ├── docker compose up -d --build --force-recreate
│   ├── Wait 30 seconds
│   └── Health check (http://localhost/health)
│
└── Post-tasks
    ├── Configure Grafana (datasource + dashboard)
    └── Final health verification
```

🔄 Полный цикл от commit до deploy

```
Developer: git push origin main
        │
        ▼
  GitHub Actions запускается
        │
  ┌─────▼─────┐      FAIL      ┌──────────┐
  │  ТЕСТЫ    │──────────────▶│ СТОП ❌  │
  │  pytest   │                │ (нет     │
  └───────────┘                │ деплоя)  │
        │                      └──────────┘
        │ PASS ✅
        ▼
  ┌──────────┐
  │  BUILD   │
  │  Docker  │──▶ Docker Hub
  └──────────┘
        │
        ▼
  ┌──────────────────────────────┐
  │  ANSIBLE PLAYBOOK            │
  │  SSH → 18.156.160.162        │
  │  ├── docker install ✅       │
  │  ├── git pull ✅             │
  │  ├── docker compose up ✅    │
  │  ├── grafana config ✅       │
  │  └── health check ✅         │
  └──────────────────────────────┘
        │
        ▼
  ┌──────────┐
  │ VERIFY   │
  │ curl ✅  │
  └──────────┘
        │
        ▼
  ✅ Pipeline GREEN — деплой завершён!
```

📊 GitHub Secrets

Секрет	Описание	Используется в
`SERVER_HOST`	IP сервера (`18.156.160.162`)	deploy (Ansible inventory)
`SERVER_USER`	SSH пользователь (`ec2-user`)	deploy (Ansible inventory)
`SSH_PRIVATE_KEY`	Содержимое `.pem` файла	deploy (SSH auth)
`DOCKER_USERNAME`	Docker Hub логин	build + deploy
`DOCKER_PASSWORD`	Docker Hub пароль/токен	build

🎯 Что сказать на защите

> “CI/CD пайплайн полностью автоматизирован через GitHub Actions. При push в main: сначала запускаются unit-тесты, затем собирается Docker образ и пушится в Docker Hub, и наконец **Ansible** подключается к серверу по SSH и выполняет деплой — устанавливает Docker если нужно, обновляет код, пересоздаёт контейнеры и проверяет health. Ansible используется вместо ручных SSH-команд, что делает деплой идемпотентным, переиспользуемым и стандартизированным.”

---

4. Terraform (Infrastructure as Code)

📁 Файлы

```
terraform/
├── main.tf              # Основной конфиг: EC2, SG, EIP, Key Pair
├── variables.tf         # Переменные: регион, тип, AMI, CIDR
├── outputs.tf           # Выходные данные: IP, SSH, URLs
├── terraform.tfstate    # Текущее состояние инфраструктуры
└── my-devops-key.pem    # SSH ключ (генерируется автоматически)
```

🎯 Что это?

Terraform — инструмент Infrastructure as Code (IaC). Описывает инфраструктуру AWS в виде кода (файлы `.tf`) и управляет её жизненным циклом: создание, изменение, удаление.

🏗️ Что создаёт Terraform

```
┌──────────────────────── AWS eu-central-1 ────────────────────────┐
│                                                                   │
│  ┌── tls_private_key ──┐      ┌── aws_key_pair ──────────────┐   │
│  │ RSA 4096 bit        │──────▶│ my-devops-key                │   │
│  │ (генерация SSH)     │      │ (загрузка в AWS)             │   │
│  └─────────────────────┘      └──────────────────────────────┘   │
│                                                                   │
│  ┌── aws_security_group ──────────────────────────────────────┐   │
│  │ health-dashboard-sg                                         │   │
│  │  Inbound:                                                   │   │
│  │  ├── 22/tcp   SSH                                           │   │
│  │  ├── 80/tcp   HTTP (Nginx)                                  │   │
│  │  ├── 443/tcp  HTTPS                                         │   │
│  │  ├── 3000/tcp Grafana                                       │   │
│  │  ├── 5000/tcp Flask App                                     │   │
│  │  └── 9090/tcp Prometheus                                    │   │
│  │  Outbound: all traffic                                      │   │
│  └─────────────────────────────────────────────────────────────┘   │
│                                                                   │
│  ┌── aws_instance ────────────────────────────────────────────┐   │
│  │ health-dashboard-server                                     │   │
│  │  AMI: Amazon Linux 2023 (latest)                            │   │
│  │  Type: t2.micro (free tier)                                 │   │
│  │  Disk: 30 GB gp3                                            │   │
│  │  user_data:                                                 │   │
│  │  ├── yum update                                             │   │
│  │  ├── install Docker + Git                                   │   │
│  │  ├── install Docker Compose (plugin + standalone)           │   │
│  │  ├── install Docker Buildx                                  │   │
│  │  └── add ec2-user to docker group                           │   │
│  └─────────────────────────────────────────────────────────────┘   │
│                                                                   │
│  ┌── aws_eip + aws_eip_association ──────────────────────────┐   │
│  │ Elastic IP: 18.156.160.162                                  │   │
│  │ (статический IP, привязан к EC2)                            │   │
│  └─────────────────────────────────────────────────────────────┘   │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

🔍 Разбор ресурсов в main.tf

Ресурс	Terraform Type	Описание
SSH ключ (генерация)	`tls_private_key`	RSA 4096 бит, генерируется Terraform
SSH ключ (AWS)	`aws_key_pair`	Публичный ключ загружается в AWS
Firewall	`aws_security_group`	6 правил inbound + outbound all
Сервер	`aws_instance`	EC2 t2.micro, Amazon Linux 2023
Статический IP	`aws_eip` + `aws_eip_association`	Elastic IP привязан к EC2

🔄 Terraform Workflow

```
terraform init      # Скачать провайдеры (AWS, TLS)
       │
       ▼
terraform plan      # Показать что будет создано/изменено
       │
       ▼
terraform apply     # Создать/изменить ресурсы в AWS
       │
       ▼
terraform output    # Показать IP, SSH команду, URLs
```

📊 Terraform State

terraform.tfstate — JSON файл, где Terraform хранит информацию о каждом созданном ресурсе. Без него Terraform не знает что уже создано и будет создавать дубликаты.

```
⚠️ ВАЖНО: НЕ удаляйте terraform.tfstate!
⚠️ ВАЖНО: НЕ редактируйте terraform.tfstate вручную!
```

🎯 Что сказать на защите

> “Terraform описывает всю AWS инфраструктуру как код. В файле main.tf определены: EC2 инстанс с Amazon Linux 2023, Security Group с нужными портами, Elastic IP для стабильного адреса, и SSH ключ. User_data автоматически устанавливает Docker при первом запуске. Terraform хранит состояние в tfstate файле и при повторном apply не создаёт дубликаты — это называется идемпотентность.”

---

5. Ansible (Configuration Management)

📁 Файлы

```
ansible/
├── inventory.ini              # Список серверов (IP + способ подключения)
├── playbook.yml               # Главный плейбук (роли + post_tasks)
└── roles/
    ├── docker/
    │   └── tasks/main.yml     # Роль: установка Docker
    └── app/
        └── tasks/main.yml     # Роль: деплой приложения
```

🎯 Что это?

Ansible — инструмент управления конфигурацией серверов. Подключается по SSH и выполняет задачи (tasks) описанные в YAML. В проекте используется **реально** — в CI/CD пайплайне для деплоя при каждом push в main.

🏗️ Как Ansible используется в проекте

```
GitHub Actions (ci-cd.yml)
        │
        ├── pip install ansible
        ├── Создаёт inventory.ini:
        │   [webservers]
        │   production ansible_host=18.156.160.162
        │              ansible_user=ec2-user
        │              ansible_ssh_private_key_file=~/.ssh/deploy_key
        │
        └── ansible-playbook -i ansible/inventory.ini ansible/playbook.yml
                │
                ▼ SSH
        ┌──────────────────────────────┐
        │  EC2: 18.156.160.162         │
        │                              │
        │  Role: docker                │
        │  ├── yum install docker      │
        │  ├── systemctl start docker  │
        │  ├── usermod -aG docker      │
        │  └── install docker-compose  │
        │                              │
        │  Role: app                   │
        │  ├── git clone/pull          │
        │  ├── cp .env.example .env    │
        │  ├── docker compose down     │
        │  ├── docker compose pull     │
        │  ├── docker compose up -d    │
        │  └── health check            │
        │                              │
        │  Post-tasks:                 │
        │  ├── Grafana config          │
        │  └── Final verification      │
        └──────────────────────────────┘
```

🔍 Role: docker — установка Docker

```
# ansible/roles/docker/tasks/main.yml
Задачи:
1. Install Docker + pip          # yum install docker python3-pip
2. Start & enable Docker         # systemctl start/enable docker
3. Add ec2-user to docker group  # Чтобы запускать без sudo
4. Check Docker Compose          # Проверяет plugin и standalone
5. Install Docker Compose        # Скачивает бинарник если нет
6. Verify Docker                 # docker --version
```

🔍 Role: app — деплой приложения

```
# ansible/roles/app/tasks/main.yml
Задачи:
1. Install git                   # yum install git
2. Create /opt/health-dashboard  # mkdir + chown ec2-user
3. Clone/update repository       # git clone / git pull (force)
4. Generate .env                 # cp .env.example .env
5. Stop old containers           # docker compose down
6. Pull pre-built images         # docker compose pull --ignore-buildable
7. Build and start               # docker compose up -d --build --force-recreate
8. Wait 30 seconds               # Ожидание инициализации
9. Show container status         # docker compose ps
10. Health check                 # curl http://localhost/health (retries: 15)
```

📊 Ansible vs SSH-скрипты

Критерий	Ручные SSH-команды	Ansible
Идемпотентность	❌ Нет	✅ Да (безопасно повторять)
Масштабируемость	❌ 1 сервер	✅ Сотни серверов
Документация	❌ Скрипт не очевиден	✅ YAML читается как документ
Роли	❌ Всё в одном	✅ Переиспользуемые роли
Ошибки	❌ Всё падает	✅ Retry, ignore_errors
В проекте	Старый подход	✅ Текущий подход (ci-cd.yml)

🎯 Что сказать на защите

> “Ansible используется в CI/CD пайплайне для деплоя. Вместо ручных SSH-команд, GitHub Actions устанавливает Ansible, генерирует inventory с IP сервера, и запускает playbook. Playbook состоит из двух ролей: docker (установка Docker) и app (деплой приложения через Docker Compose). Это обеспечивает идемпотентность — можно запускать повторно без проблем, а также масштабируемость — достаточно добавить серверы в inventory.”

---

6. Kubernetes (Container Orchestration)

📁 Файлы

```
k8s/
├── namespace.yaml       # Namespace: health-dashboard
├── configmap.yaml       # Конфигурация (DATABASE_URL, REDIS_URL)
├── secret.yaml          # Секреты (пароли в base64)
├── deployment.yaml      # Deployment: 2 replicas Flask app
├── service.yaml         # Service: LoadBalancer :80 → :5000
└── helm/
    └── health-dashboard/
        ├── Chart.yaml         # Helm chart metadata
        ├── values.yaml        # Параметры по умолчанию
        └── templates/         # Шаблоны манифестов
            ├── namespace.yaml
            ├── configmap.yaml
            ├── secret.yaml
            ├── deployment.yaml
            └── service.yaml
```

🎯 Что это?

Kubernetes (K8s) — платформа оркестрации контейнеров. В проекте подготовлены манифесты для деплоя в K8s кластер. **На текущем этапе не используется в production** — приложение работает через Docker Compose на EC2.

🏗️ Kubernetes ресурсы

```
┌──────────── Namespace: health-dashboard ────────────┐
│                                                     │
│  ConfigMap                  Secret                  │
│  ├── DATABASE_URL           ├── POSTGRES_PASSWORD   │
│  ├── REDIS_URL              └── SECRET_KEY          │
│  └── FLASK_ENV                                      │
│                                                     │
│  Deployment (2 replicas)                            │
│  ├── Pod 1: Flask app                               │
│  └── Pod 2: Flask app                               │
│                                                     │
│  Service (LoadBalancer)                             │
│  └── :80 → :5000                                    │
│                                                     │
└─────────────────────────────────────────────────────┘
```

🎯 Что сказать на защите

> “Kubernetes манифесты подготовлены для будущего масштабирования. Есть Deployment с 2 репликами Flask-приложения, Service типа LoadBalancer, ConfigMap и Secret для конфигурации. Также создан Helm chart для параметризированного деплоя. На текущем этапе используется Docker Compose, но инфраструктура готова к миграции в K8s кластер.”

7. Monitoring (Prometheus + Grafana)

📁 Файлы

```
monitoring/
├── prometheus.yml                          # Конфигурация Prometheus
├── alert_rules.yml                         # Правила алертов
├── grafana/
│   └── provisioning/
│       └── datasources/datasources.yaml    # Автоконфигурация Prometheus
└── loki-config.yaml                        # Конфигурация Loki (не используется)

grafana/
├── provisioning/
│   ├── dashboards/
│   │   ├── dashboards.yml                  # Dashboard provisioning
│   │   └── health-dashboard.json           # Готовый dashboard
│   └── datasources/
│       └── datasources.yml                 # Datasource provisioning
├── dashboard.json                          # Dashboard (backup)
├── application-dashboard.json              # App-specific dashboard
└── working-dashboard.json                  # Рабочий dashboard

scripts/
├── configure_grafana.sh                    # Скрипт настройки Grafana API
└── verify_monitoring.sh                    # Скрипт проверки мониторинга
```

🎯 Что это?

Prometheus собирает метрики из Flask-приложения каждые 60 секунд. **Grafana** визуализирует эти метрики в красивых дашбордах.

🏗️ Архитектура мониторинга

```
┌──────────────────────────────────────────────────────────┐
│                  MONITORING STACK                        │
│                                                          │
│  Flask App (:5000)                                       │
│  └── /metrics endpoint                                   │
│       ├── request_count_total                            │
│       ├── request_latency_seconds                        │
│       ├── cpu_usage_percent                              │
│       ├── memory_usage_percent                           │
│       └── disk_usage_percent                             │
│              │                                           │
│              │ scrape каждые 60s                         │
│              ▼                                           │
│  Prometheus (:9090)                                      │
│  ├── Хранит метрики (retention: 3h)                      │
│  ├── PromQL запросы                                      │
│  └── Alert rules                                         │
│              │                                           │
│              │ datasource                                │
│              ▼                                           │
│  Grafana (:3000)                                         │
│  ├── Health Dashboard                                    │
│  ├── Application Dashboard                               │
│  └── Credentials из .env                                 │
│     (GF_SECURITY_ADMIN_USER / GF_SECURITY_ADMIN_PASSWORD)│
│                                                          │
└──────────────────────────────────────────────────────────┘
```

🔍 Prometheus конфигурация

```yaml
# monitoring/prometheus.yml
global:
  scrape_interval: 60s         # Опрос каждые 60 секунд
  evaluation_interval: 60s     # Проверка правил каждые 60 секунд

scrape_configs:
  - job_name: 'prometheus'     # Мониторинг самого себя
    targets: ['prometheus:9090']

  - job_name: 'flask-app'     # Мониторинг Flask
    targets: ['app:5000']      # Обращается по имени контейнера
```

📊 Доступ к мониторингу

Сервис	URL	Логин
Grafana	http://18.156.160.162:3000	Из `.env` (`GF_SECURITY_ADMIN_USER` / `GF_SECURITY_ADMIN_PASSWORD`)
Prometheus	http://18.156.160.162:9090	Без авторизации
Prometheus Targets	http://18.156.160.162:9090/targets	flask-app должен быть UP

🎯 Что сказать на защите

> “Мониторинг построен на связке Prometheus + Grafana. Prometheus опрашивает Flask-приложение каждые 60 секунд через endpoint /metrics и собирает метрики: количество запросов, время ответа, загрузку CPU, памяти и диска. Grafana подключается к Prometheus как datasource и визуализирует метрики в реальном времени. Настройка Grafana автоматизирована через скрипт configure_grafana.sh, который запускается при деплое.”

---

8. Testing (Unit Tests)

📁 Файлы

```
tests/
├── __init__.py        # Python package init
├── conftest.py        # Pytest fixtures (создаёт test client)
└── test_app.py        # Основные тесты приложения
    test_health.py     # Тесты health endpoint
```

🎯 Что это?

Unit-тесты — автоматическая проверка работоспособности приложения. Запускаются в CI/CD пайплайне **перед** деплоем.

🏗️ Что тестируется

```
tests/test_app.py
├── test_health_endpoint         # GET /health → 200 + JSON
├── test_index_endpoint          # GET / → 200
├── test_metrics_endpoint        # GET /metrics → 200 + prometheus
└── test_system_info             # GET /api/system-info → 200 + JSON

tests/test_health.py
├── test_health_status           # Проверка поля "status"
├── test_health_has_cpu          # Проверка наличия CPU метрики
├── test_health_has_memory       # Проверка наличия memory метрики
└── test_health_response_time    # Время ответа < 1 секунда
```

🔄 Как запускаются тесты

```
# Локально:
pytest tests/ -v --cov=app --cov-report=term-missing

# В CI/CD (GitHub Actions):
# Stage 1 (test job) запускает ту же команду
# Если хотя бы 1 тест FAIL → весь пайплайн СТОП
```

📊 Coverage (покрытие кода)

Coverage показывает какой процент кода app/ покрыт тестами. `--cov-report=term-missing` показывает конкретные строки, которые не были протестированы.

🎯 Что сказать на защите

> “Проект включает unit-тесты на pytest. Тестируются все endpoints: health, metrics, system-info и главная страница. Тесты запускаются автоматически в CI/CD перед каждым деплоем. Если хотя бы один тест не проходит — деплой блокируется. Также измеряется code coverage — процент кода, покрытого тестами.”

---

9. Nginx (Reverse Proxy)

📁 Файл: `nginx/nginx.conf`

🎯 Что это?

Nginx — reverse proxy сервер. Принимает запросы на порт 80 и перенаправляет их на Flask приложение (порт 5000). Добавляет security headers.

📋 Конфигурация

```
# nginx/nginx.conf

upstream flask_app {
    server app:5000;           # Обращение к Flask по имени контейнера
}

server {
    listen 80;                 # Слушает HTTP
    server_name _;             # Любой домен

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    location / {               # Все запросы → Flask
        proxy_pass http://flask_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /metrics {        # Prometheus метрики
        proxy_pass http://flask_app/metrics;
    }

    location /nginx-health {   # Health check самого Nginx
        return 200 'OK';
    }
}
```

🔄 Request Flow

```
Пользователь
    │
    │  http://18.156.160.162/health
    ▼
┌─────────┐
│  Nginx  │ :80
│ (proxy) │
│ + security headers
└─────────┘
     │
     │ proxy_pass
     ▼
┌─────────┐
│  Flask  │ :5000
│  app.py │
└─────────┘
     │
     ▼
  JSON response
```

🎯 Что сказать на защите

> “Nginx работает как reverse proxy — принимает запросы на порт 80 и проксирует их на Flask приложение на порте 5000. Это стандартная production практика: Nginx лучше справляется с высокой нагрузкой, добавляет security headers (X-Frame-Options, X-Content-Type-Options, X-XSS-Protection), и в будущем может обрабатывать SSL/HTTPS.”

---

10. Disaster Recovery (Infrastructure Recovery)

📁 Файл: `.github/workflows/infrastructure-recovery.yml`

🎯 Что это?

Disaster Recovery пайплайн — полное пересоздание инфраструктуры AWS с нуля. Используется ТОЛЬКО при катастрофе — когда EC2 инстанс потерян или недоступен.

⚠️ Важные моменты

- Запускается **вручную** (кнопка “Run workflow”)
- **Уничтожает** ВСЕ текущие AWS ресурсы и создаёт новые
- **Новый Elastic IP** — при каждом запуске
- После запуска нужно обновить `SERVER_HOST` в GitHub Secrets
- Использует **Terraform** для создания инфраструктуры

🔄 Пошаговый процесс

```
Admin нажимает "Run workflow"
            │
            ▼
┌──────────────────────────────────────────────────┐
│ Step 1: CLEANUP (AWS CLI)                        │
│ ├── Terminate EC2 instances (tag: my-devops-project)│
│ ├── Delete Key Pair (my-devops-key)              │
│ ├── Release Elastic IPs (tag: health-dashboard-eip) │
│ └── Delete Security Group (health-dashboard-sg)  │
└──────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────┐
│ Step 2: TERRAFORM (terraform apply)              │
│ ├── Key Pair (SSH ключ)                          │
│ ├── Security Group (порты)                       │
│ ├── EC2 Instance (Amazon Linux 2023 + Docker)    │
│ └── Elastic IP (НОВЫЙ!)                          │
└──────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────┐
│ Step 3: WAIT 90 seconds (boot)                   │
└──────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────┐
│ Step 4: DEPLOY (SSH)                             │
│ ├── git clone в /opt/health-dashboard            │
│ ├── Создание .env                                │
│ ├── docker compose up -d --build                 │
│ ├── Настройка Grafana                            │
│ └── docker compose ps                            │
└──────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────┐
│ Step 5: VERIFY                                   │
│ ├── curl http://NEW_IP — Nginx                   │
│ ├── curl http://NEW_IP:3000 — Grafana            │
│ └── curl http://NEW_IP:9090 — Prometheus         │
└──────────────────────────────────────────────────┘
                      │
                      ▼
    ⚠️ Обновить SERVER_HOST в GitHub Secrets!
```

📊 Что произошло с нашим проектом

```
HISTORY:
──────────────────────────────────────────────────────
Первоначальный сервер:   IP 3.127.155.114    ← ПОТЕРЯН (Elastic IP released)
Infrastructure Recovery: IP 18.156.160.162   ← ТЕКУЩИЙ (новый Elastic IP)
EC2 Instance:           i-059c8320d831be2bf
──────────────────────────────────────────────────────

⚠️ Elastic IP нельзя восстановить после release!
   AWS не позволяет запросить конкретный IP.
```

🎯 Что сказать на защите

> “В проекте есть пайплайн disaster recovery для полного восстановления инфраструктуры. Если сервер потерян — нажимаем кнопку в GitHub Actions, и пайплайн через AWS CLI очищает старые ресурсы, через Terraform создаёт новый EC2, Security Group и Elastic IP, затем деплоит приложение. Всё восстанавливается автоматически за несколько минут.”

✅ Итого: Все этапы проекта

```
┌──────────────────────────────────────────────────────────────────┐
│                    HEALTH DASHBOARD PROJECT                      │
│                    Elastic IP: 18.156.160.162                    │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  📝 КОД          Flask + Python          → app/app.py            │
│  🧪 ТЕСТЫ        pytest + coverage       → tests/                │
│  🐳 КОНТЕЙНЕРЫ   Docker Compose (6 шт)   → docker-compose.yml    │
│  🔄 CI/CD        GitHub Actions + Ansible → .github/workflows/   │
│  🏗️ IaC          Terraform (AWS)         → terraform/            │
│  ⚙️ CONFIG       Ansible (roles)         → ansible/              │
│  📊 МОНИТОРИНГ   Prometheus + Grafana     → monitoring/          │
│  🌐 PROXY        Nginx                   → nginx/                │
│  ☸️ K8s READY    Kubernetes + Helm        → k8s/                 │
│  🔥 DR           Infrastructure Recovery  → infrastructure-recovery │
│                                                                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ПАЙПЛАЙН ДЕПЛОЯ (ci-cd.yml):                                    │
│  push → test → Docker build → ANSIBLE deploy → verify            │
│                                                                  │
│  ПАЙПЛАЙН ВОССТАНОВЛЕНИЯ (infrastructure-recovery.yml):         │
│  button → cleanup → TERRAFORM apply → SSH deploy → verify        │
│                                                                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ДОСТУП:                                                         │
│  🌐 Приложение:  http://18.156.160.162                           │
│  📊 Grafana:     http://18.156.160.162:3000                      │
│  🔥 Prometheus:  http://18.156.160.162:9090                      │
│  🏥 Health:      http://18.156.160.162/health                    │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Terraform vs Ansible — кто что делает

```
TERRAFORM                              ANSIBLE
═══════════                            ═══════
"ЧТО создать"                         "КАК настроить"

├── EC2 Instance                       ├── Установка Docker
├── Security Group                     ├── Git clone/pull
├── Elastic IP                         ├── Docker Compose up
├── Key Pair                           ├── Grafana настройка
└── user_data (bootstrap)              └── Health checks

Когда: сервер потерян                  Когда: каждый деплой
Пайплайн: infrastructure-recovery     Пайплайн: ci-cd
Уровень: облако AWS                   Уровень: ОС + приложение
```